Von Neumann X Harvard

Monday, April 20, 2026 8:00 PM

Além do seu conjunto de instruções, processadores são classificados pela forma como lidam com suas próprias instruções em memória (como processam dados)

Arquitetura Von Neumann

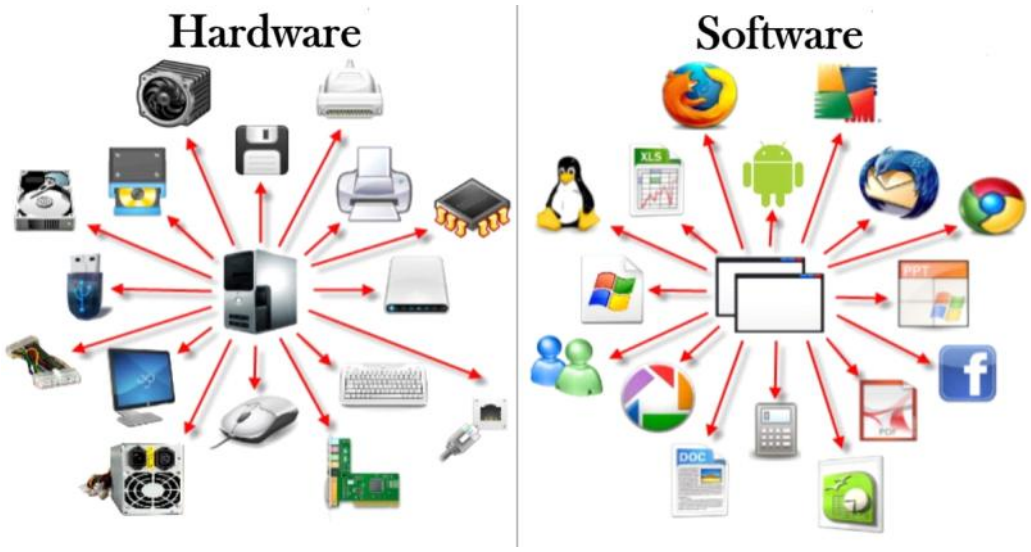
- Modelo clássico de computação (1945)
- Proposta por John von Neumann
  - Dados e instruções compartilham a mesma memória
  - Uso de um único barramento
  - Execução sequencial de instruções
- Vantagem: simplicidade
- Desvantagem: gargalo de comunicação cpu-memória

Arquitetura Harvard

- Memória separada para dados e instruções
- Dois barramentos independentes
- Acesso simultâneo a dados e instruções
- Vantagem: Maior desempenho
- Desvantagem: Complexidade de implementação
- Aparecem muito mais em microcontroladores e sistemas embarcados

Componentes Computador

Monday, April 20, 2026 6:37 PM



Software

- Software Básico
  - Gerenciamento de operação do computador
- Software Aplicativo
  - Auxílio para o usuário

- Modo Núcleo:
  - acesso ao hardware
- Modo Usuário:
  - subconjunto de instruções apenas

Sistema Operacional

Monday, April 20, 2026 6:42 PM

- Utilizar recursos de maneira otimizada é muito complexo
- Essa é a função do Sistema Operacional

Ideia do Objetivo de um Sistema Operacional

Fornecer aos programas do usuário um modelo do computador melhor, mais simples e mais limpo, assim como lidar com o gerenciamento de todos os recursos mencionados

Costuma estar relacionado com duas funções:

- Máquina Estendida
  - Esconde detalhes de execução
  - Interface amigável com o usuário
  - Mais fácil de utilizar

- Gerenciador de Recursos
  - Controle do funcionamento do computador
  - Gerenciamento de recursos (memória, CPU, etc)

Qual a Função de um Sistema Operacional?

1. Facilitar o acesso aos recursos do sistema
2. Compartilhar recursos de forma organizada
3. Compartilhar recursos de forma protegida

O cliente do SO são os programas aplicativos

Sistema Operacional como Gerenciador e Recursos

- Fornecer alocação ordenada e controlada de recursos computacionais
- As duas funções do SO podem ser vistas como:

- Abstrações para programas é uma visão top-down
- Gerenciar partes de um sistema complexo é uma visão bottom-up

Principal função do SO é manter o controle de recurso e mediar conflitos.

- Gerenciamento resulta em compartilhamento (multiplexação):
  - No tempo: Programas ou usuários aguardando recurso
  - **Exemplo:** programas executando ao mesmo tempo
  - No espaço: Programas utilizando um recurso simultaneamente
  - **Exemplo:** memória

O que os Sistemas Operacionais Controlam?

- Para cada recurso da máquina, o SO:
  - Mantém informações sobre o recurso
  - Decide quem pode acessar o recurso
  - Aloca o recurso
  - Libera o recurso

- No que tange ao controle do recurso, o SO:
  - Ser eficiente (maximizar a utilização)
  - Ser justo (possuir um tempo de resposta aceitável)

SO X Software Normal

Monday, April 20, 2026 6:46 PM

Software no modo usuário pode ser substituído

- Exemplo: troca de navegador

- Não é possível trocar o tratador de interrupção do relógio do sistema
- Porém, às vezes essa definição pode se perder:

1. Sistemas Embarcados

Em dispositivos muito específicos (como o painel de um micro-ondas, o freio ABS de um carro ou um roteador simples), o hardware tem poucos recursos (pouca RAM e CPU fraca). Por isso, não há o luxo de separar Modo Usuário e Modo Núcleo. O Sistema Operacional e o programa aplicativo são compilados juntos em um único bloco de código. Se o "aplicativo" falhar, o sistema inteiro trava.

2. Sistemas de Arquivos (modo usuário)

Tradicionalmente, gerenciar o disco e os arquivos é uma tarefa superprivilegiada que fica exclusivamente dentro do Modo Núcleo. Porém, sistemas modernos permitem criar Sistemas de Arquivos no Espaço do Usuário (conhecido como FUSE no Linux). É o que acontece quando você usa um programa como o Google Drive para Desktop ou monta uma pasta na nuvem: um programa comum de usuário está executando uma função que, na teoria, deveria ser exclusiva do Sistema Operacional.

3. Funções Privilegiadas (troca de senha)

Esse é um caso clássico de segurança. No Linux: o arquivo que guarda as senhas (`/etc/shadow`) é extremamente protegido e só o "superusuário" (root/núcleo) pode alterá-lo. Mas você, como um usuário comum sem privilégios, precisa ter o direito de trocar a *sua* própria senha. Para resolver isso, o programa que troca a senha (o comando `passwd`) é um programa de Modo Usuário, mas ele recebe temporariamente privilégios de Modo Núcleo apenas durante a sua execução (uma técnica chamada *Setuid bit*). A definição se perde aqui porque você tem um aplicativo de usuário comum manipulando arquivos do núcleo do sistema.

Ciclo Clock

Monday, April 20, 2026 8:09 PM

1. Sincronização (O Maestro)

Dentro do processador, existem bilhões de transistores. Se cada um trabalhasse na sua própria velocidade, os sinais elétricos chegariam em tempos diferentes, causando um caos de dados (um bit chegaria antes do outro e a soma daria errado).

O clock define o momento exato em que todos os circuitos devem "parar e ler" o sinal elétrico atual. Isso garante que o dado esteja **estável** antes de ser passado para a próxima etapa.

2. Segmentação (O Pipeline)

Se você tem uma instrução que leva 5 ciclos, o processador não espera ela terminar para começar a próxima.

Graças ao clock, ele divide a instrução em fatias (etapas do ciclo que você viu na imagem anterior). A cada "batida" do clock:

- A Instrução 1 passa para a etapa de **Execução**.
- A Instrução 2 passa para a etapa de **Decodificação**.
- A Instrução 3 entra na etapa de **Busca**.

Processadores

Monday, April 20, 2026 7:50 PM

O processador (ou CPU) é o componente que executa as instruções dos programas, mas ele não "entende" linguagens de alto nível como Python ou C++ diretamente.

- Cada CPU tem um conjunto específico de instruções

Ciclo de Instrução de um processador, também conhecido como ciclo *Fetch-Decode-Execute*. É o processo repetitivo que a CPU realiza bilhões de vezes por segundo para processar cada linha de código de um programa.

Em processadores modernos, utiliza-se o **Pipeline**: enquanto uma instrução está na fase de "Execução", o processador já está "Buscando" a próxima e "Decodificando" uma terceira. Isso permite que, embora uma instrução leve vários passos para concluir, o processador entregue um resultado a cada ciclo de clock.

1. Busca Instrução (Fetch)

O processador consulta o registrador **IP** (Instruction Pointer) ou **PC** (Program Counter) para saber o endereço da próxima instrução na memória RAM. Ele "busca" essa instrução e a traz para dentro do processador.

2. Decodifica Instrução (Decode)

A instrução que veio da memória é apenas um número binário (opcode). Nesta fase, a unidade de controle do processador interpreta esse número para entender o que deve ser feito: é uma soma? Uma movimentação de dados? Um salto para outra parte do programa?

3. Busca Operando (Fetch Operand)

Muitas instruções precisam de dados extras para serem executadas.

- **Exemplo:** Se a instrução for "Somar 5 ao valor da memória X", o processador precisa buscar o valor que está no endereço X antes de realizar a conta.

4. Executa Instrução (Execute)

Aqui o trabalho é feito de fato. A **ULA (Unidade Lógica e Aritmética)** entra em ação para realizar cálculos matemáticos ou operações lógicas baseadas no que foi decodificado.

5. Armazena Resultado (Write-back)

O resultado da execução é gravado. Pode ser guardado em um **registrador** interno do processador (para uso imediato na próxima linha de código) ou enviado de volta para a **memória RAM**.

Técnica de hardware para busca de uma ou mais instruções além da próxima a ser executada, ao mesmo tempo.

O processador obtenção serviços do SO ocorre por meio de:

- Chamadas de Sistema
  - Instruções **TRAP**

O programa passa o controle para o SO que faz o acesso restrito

Interrupção

Monday, April 20, 2026 10:33 PM

Uma interrupção é um evento externo que causa o processador parar a execução do programa corrente e desviar a execução para um bloco de código chamado rotina de interrupção.

A **Interrupção** é um dos mecanismos mais fundamentais da computação. Sem ela, o processador seria "egoísta", ficando preso em uma tarefa e ignorando tudo ao seu redor, ou perderia um tempo precioso perguntando a cada segundo para o teclado: *"Você digitou algo agora? E agora?"* (técnica conhecida como *Polling*).

O Fluxo de uma Interrupção

Quando um hardware (como o teclado ou a placa de rede) precisa de atenção, o fluxo segue estes passos:

1. **O Sinal:** O controlador do dispositivo envia um sinal elétrico para um pino específico do processador (IRQ - *Interrupt Request*).
2. **A Pausa:** O processador termina a instrução atual que estava executando.
3. **Salvamento de Estado:** Para não perder o que estava fazendo, o processador salva o valor atual do **Contador de Programa (PC)** e dos **Registradores** na pilha (Stack).
4. **O Desvio:** O processador consulta uma tabela na memória chamada **Vetor de Interrupções**. Essa tabela contém o endereço de onde começa o código para lidar com aquele evento específico.
5. **A Rotina (ISR):** O processador executa a **Rotina de Serviço de Interrupção** (ISR - *Interrupt Service Routine*).
6. **O Retorno:** Após terminar a rotina, o processador recupera os valores salvos da pilha e volta a executar o programa exatamente de onde parou.

Sem interrupções, um programa travado em um "loop infinito" congelaria o computador inteiro para sempre.

Registradores Internos

Monday, April 20, 2026 8:19 PM

Como buscar um dado na RAM pode levar centenas de ciclos de clock, o processador utiliza esses registradores internos como uma "mesa de trabalho" imediata.

1. Contador de Programa (PC - Program Counter)

- **O que faz:** Armazena o endereço de memória da **próxima** instrução que o processador deve buscar.
- **No x86-64:** Ele é chamado de **RIP** (Instruction Pointer).
- **Importância:** Sem ele, o processador "se perderia". Quando ocorre um desvio (um if ou um loop), o SO ou o hardware alteram o valor do PC para apontar para o novo endereço de código.

2. Ponteiro de Pilha (SP - Stack Pointer)

- **O que faz:** Aponta para o topo da **Pilha (Stack)** na memória.
- **No x86-64:** Ele é chamado de **RSP**.
- **Importância:** A pilha é usada para armazenar variáveis locais e, principalmente, o **endereço de retorno** de funções. Quando você chama uma função, o endereço atual é "empilhado" (push) e o SP é atualizado. Quando a função termina, o endereço é "desempilhado" (pop) para o processador saber para onde voltar.

3. Program Status Word (PSW)

- **O que faz:** É um registrador de flags (bandeiras). Ele não guarda números para cálculos, mas sim **bits de estado** sobre a última operação realizada e o estado atual da CPU.
- **Flags Comuns:**
- **Zero Flag (Z):** Ativado se o resultado da última conta foi zero.
- **Carry Flag (C):** Ativado se houve um "vai um" em uma conta aritmética.
- **Negative Flag (N):** Ativado se o resultado foi negativo.
- **Modo de Execução:** Um bit que indica se a CPU está em **Modo Usuário** ou **Modo Kernel**.

CPUs possuem dois modos de execução:

- Um bit do PSW indica o modo de uso
- Modo Núcleo: tudo liberado (instruções e recursos)
- Modo Usuário: possui restrições a alguns recursos

Normalmente E/S e proteção de memória são proibidos

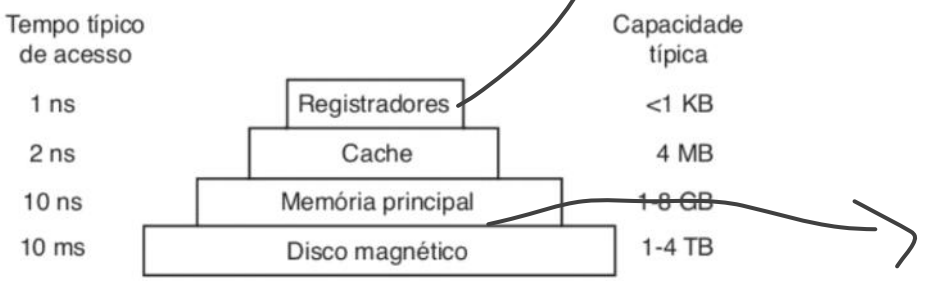
Hierarquia Memória

Monday, April 20, 2026 9:58 PM

Tem que ser rápida, muito rápida

- Tão rápida quanto a CPU necessita
- Na prática, esse objetivo não é cumprido
- Utiliza-se uma abordagem de hierarquia para compensar

Os registradores permitem que o processador mantenha os dados mais importantes "à mão", reduzindo a necessidade de trafegar informações pelo barramento externo.



RAM, armazena dados dos programas em execução

Memória Cache

- Cache é controlada pelo hardware (linhas de cache)
- Caching é o ato de trazer para perto aquilo que é mais utilizado
- O processador decide automaticamente o que deve estar na cache com base no que foi usado recentemente.

A eficácia da cache vem de dois fatores:

- Localidade temporal - o que foi usado pode ser usado de novo
- Localidade espacial - ao usar algo, grandes chances de

também usar algo próximo a ele

Discos:

A principal diferença entre o **HD (Hard Disk Drive)** e o **SSD (Solid State Drive)** está na tecnologia que eles usam para armazenar e ler dados: o HD é **mecânico**, enquanto o SSD é **eletrônico**.

Memória Virtual

Monday, April 20, 2026 10:21 PM

1. O Conceito de Mapeamento (Paginação)

Cada programa (processo) opera em seu próprio **Espaço de Endereçamento Virtual**. O Sistema Operacional divide esse espaço e a memória RAM física em blocos de tamanho fixo chamados **Páginas** (no virtual) e **Molduras/Frames** (no físico).

O SO mantém uma **Tabela de Páginas** para cada processo. Essa tabela funciona como um "mapa" que diz: "A Página 1 do Programa A está na Moldura 50 da RAM física".

2. Como ocorre o Compartilhamento

Para compartilhar memória entre dois programas (Programa A e Programa B), o Sistema Operacional simplesmente aponta uma entrada na Tabela de Páginas de cada um para a **mesma moldura física** na RAM.

- **Programa A:** Acessa seu endereço virtual 0x1000. A Tabela de Páginas aponta para o **Frame 500** na RAM.
- **Programa B:** Acessa seu endereço virtual 0x5000. A Tabela de Páginas também aponta para o **Frame 500** na RAM.

Embora os endereços virtuais nos programas sejam diferentes, eles estão manipulando exatamente os mesmos bits físicos.

3. Casos de Uso Comuns

A. Bibliotecas Compartilhadas (DLLs ou .so)

Imagine que você tem 10 programas abertos que usam a biblioteca padrão do C (libc). Em vez de carregar a libc dez vezes na RAM (gastando muita memória), o SO carrega uma única cópia física e mapeia o endereço virtual de todos os 10 programas para essa mesma cópia.

B. Comunicação entre Processos (IPC)

Dois processos podem querer trocar dados rapidamente. O SO reserva uma área da RAM e a mapeia nos dois processos. O que o Programa A escrever ali, o Programa B poderá ler instantaneamente, sem precisar enviar mensagens via rede ou arquivos.

4. Proteção e "Copy-on-Write" (Cópia na Escrita)

O compartilhamento levanta uma questão: e se um programa quiser apenas ler a biblioteca, mas o outro tentar alterá-la?

- **Permissões:** Cada entrada na Tabela de Páginas possui bits de proteção (**R**ead, **W**rite, e**X**ecute). O SO pode marcar a memória compartilhada como "Somente Leitura" para evitar que um programa estrague o código do outro.
- **Copy-on-Write (CoW):** Quando você usa o comando fork() para criar um processo filho, o SO não copia a memória imediatamente. Ele compartilha a memória original entre pai e filho. Somente se um deles tentar **escrever** (mudar um dado), o SO cria uma cópia real daquela página específica para o processo que tentou a alteração. Isso economiza um tempo enorme.

Entrada e Saída

Monday, April 20, 2026 10:23 PM

1. A Divisão: Dispositivo vs. Controlador

Para o computador funcionar, o hardware é separado em duas partes distintas:

- **O Dispositivo (Hardware Físico):** É a parte mecânica ou física (o motor que gira o disco, o laser que lê o CD, os sensores do mouse). Eles tendem a ser "burros", apenas seguindo ordens elétricas.
- **O Controlador (Chip/Placa):** É um chip (ou conjunto de chips) que atua como o "cérebro" daquele hardware. Ele recebe comandos lógicos do SO e os transforma em sinais elétricos para o dispositivo.

Exemplo do Braço do Disco: Quando o SO quer ler o "Arquivo X", ele não sabe em qual trilha física do disco o arquivo está. Ele pede ao **Controlador**: "Me dê os dados do setor 500". O controlador, por sua vez, sabe exatamente onde o braço deve se mover e, se houver um defeito físico naquela trilha, ele faz o **remapeamento** automático sem que o SO perceba o erro.

2. O Driver de Dispositivo (A Ponte de Software)

O **Driver** é o tradutor. Como existem milhares de fabricantes de hardware, o SO não pode vir com o código de todos eles de fábrica.

- **Interface Padrão:** O SO define uma interface padrão (ex: "todo driver de disco deve aceitar o comando *Ler Bloco*").
- **Modo Núcleo (Kernel Mode):** Como o driver precisa enviar comandos diretamente para os endereços de hardware (portas de E/S ou memória mapeada), ele **precisa** rodar em modo privilegiado. Se um driver falhar, ele pode travar o sistema inteiro (a famosa "Tela Azul").

3. Como o Driver entra no Kernel?

Existem três formas clássicas de integrar esse software ao "coração" do sistema:

4. **Religação (Estático):** Antigamente, para instalar um hardware novo, você precisava compilar o código do Kernel novamente incluindo o driver. Isso exigia reiniciar a máquina.
5. **Entrada em Arquivo de Configuração:** O SO lê uma lista de drivers necessários durante o boot (comum em sistemas tipo Unix/Linux com arquivos /etc/modules).
6. **Carregamento Dinâmico (Hot-pluggable):** É o que usamos hoje com o **USB**. O SO detecta o hardware, localiza o driver no disco e o "injeta" na memória do Kernel enquanto o computador está ligado, sem necessidade de reinicialização.

Resumo do Fluxo de Comunicação

Para você visualizar a hierarquia, o caminho de um dado é sempre este:

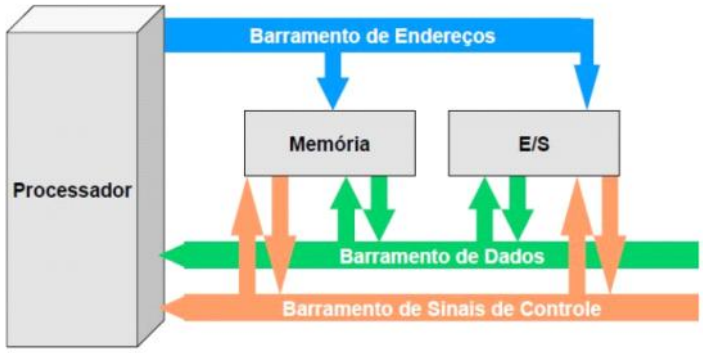
7. **Usuário:** Clica em "Salvar".
8. **SO (Kernel):** Chama a função padrão de escrita.
9. **Driver:** Traduz "Salvar" para comandos específicos da marca do SSD.
10. **Controlador:** Recebe os comandos do driver e aciona os chips de memória flash.
11. **Dispositivo:** Grava os elétrons nas células de memória.

Barramentos

Monday, April 20, 2026 10:42 PM

Definição e Funcionamento

- Um barramento é um conjunto de conexões elétricas e linhas de comunicação que permitem a interligação e o transporte de informações entre dispositivos, como a CPU, a Memória e outros periféricos.
- Ele funciona transmitindo sinais que representam valores binários.
- Uma regra fundamental é que apenas um dispositivo pode enviar conteúdo em uma linha por vez.



Estrutura do Barramento:

- **Barramento de Endereços:** Usado para indicar a origem ou o destino dos dados.
- **Barramento de Dados:** Por onde a informação em si trafega.
- **Barramento de Sinais de Controle:** Usado para gerenciar o acesso e as operações no barramento.

Fluxos de Informação:

- **Memória → Processador:** Transmissão de instruções ou dados para execução.
- **Processador → Memória:** O processador gravando dados na memória.
- **E/S → Processador:** Dados vindo de um dispositivo de Entrada/Saída (por meio de um módulo de E/S).
- **Processador → E/S:** O processador enviando dados ou comandos para um dispositivo periférico.
- **Dispositivo E/S → Memória:** Transferência direta de dados sem passar ativamente pela CPU, utilizando a técnica de DMA (Acesso Direto à Memória).

Gerenciamento e Exemplos

- Cada barramento possui suas próprias características, como funções específicas e diferentes taxas de transferência.
- É exigido que o Sistema Operacional saiba de tudo isso para poder gerenciar o hardware da melhor forma.
- Como exemplo contemporâneo, o barramento principal dos computadores modernos é o **PCIe** (*Peripheral Component Interconnect Express*).

Conceitos Gerais

Tuesday, April 21, 2026 4:57 PM

Processos

- Um processo é definido fundamentalmente como um programa em execução.
- Cada processo possui um espaço de endereçamento específico.
- Um processo detém um conjunto de recursos que inclui registradores (como PC e ponteiro de pilha), arquivos abertos, processos relacionados e alarmes pendentes.

Arquivos

- Arquivos consistem em informações que estão logicamente relacionadas.
- O conceito de arquivos é utilizado para esconder as particularidades físicas e complexidades de discos e dispositivos de Entrada e Saída (E/S).
- As operações de criar, remover, ler e escrever arquivos são feitas por meio de Chamadas de Sistema.
- O conceito de "Diretório" existe para manter e agrupar esses arquivos, sendo organizados estruturalmente em uma hierarquia de árvores.

Entrada e Saída (E/S)

- A Entrada e Saída é o elemento básico do uso de um computador, pois é por meio dela que os usuários recebem os resultados dos trabalhos requisitados.
- O Sistema Operacional possui um subsistema dedicado exclusivamente ao gerenciamento da diversidade de dispositivos de E/S existentes.
- O software que lida com a E/S pode ser tanto genérico e independente quanto específico, como é o caso dos drivers de dispositivos.

Chamadas de Sistema (System Calls)

- As Chamadas de Sistema são o mecanismo que um programa de computador utiliza para solicitar um serviço ao Sistema Operacional no qual está operando.
- O material apresenta a chamada "Read" como exemplo prático (contador = read(fd, buffer, nbytes)), a qual retorna o número de bytes lidos ou o valor -1 em caso de erro.
- O funcionamento de uma chamada de sistema exige transitar do "Espaço do usuário" para o "Espaço do núcleo" (do Sistema Operacional) por meio de uma instrução TRAP, envolvento passos como empilhamento de parâmetros, execução do tratador da chamada e retorno do controle para o programa de origem.

Espaços de Endereços

- O espaço de endereços também é listado inicialmente como um dos principais conceitos e abstrações geridos por um Sistema Operacional

Estruturas

Tuesday, April 21, 2026 5:42 PM

Sistemas Monolíticos

- É a forma de organização mais comum entre os sistemas operacionais.
- O SO funciona como um único programa executado integralmente em modo núcleo, escrito como um conjunto de rotinas agrupadas e ligadas em um único arquivo binário.
- Nessa estrutura, qualquer procedimento pode chamar livremente qualquer outro procedimento. Isso torna o sistema extremamente eficiente, mas cria um grau de liberdade que pode ser caótico.
- Sua maior desvantagem é a falta de ocultação de informações em nível de sistema (toda rotina enxerga qualquer outra rotina). Consequentemente, um erro ou quebra em qualquer parte do sistema derruba o SO inteiro.

Sistemas de Camadas

Trata-se de uma generalização da estrutura monolítica, dividindo o sistema em níveis hierárquicos.

- **Camada 5:** O operador.
- **Camada 4:** Programas de usuário.
- **Camada 3:** Gerenciamento de entrada/saída.
- **Camada 2:** Comunicação operador-processo.
- **Camada 1:** Memória e gerenciamento de tambor.
- **Camada 0:** Alocação do processador e multiprogramação.

Microkernel (Micronúcleo)

- A ideia central do microkernel é dividir o sistema em módulos pequenos e com limites e funções muito bem definidos.
- Apenas o microkernel em si é executado em modo núcleo (com privilégios totais), enquanto os outros módulos são executados em modo usuário.
- O objetivo dessa abordagem é melhorar drasticamente a confiabilidade do sistema.
- Se um driver de dispositivo, que agora roda como um processo de usuário, parar de funcionar ou apresentar erro, a falha não é fatal e não compromete o sistema inteiro.

Máquinas Virtuais

- O conceito surgiu durante a época dos sistemas em lote (*batch*), como o OS/360, motivado pela vontade dos usuários de operarem interativamente em terminais.

Exokernels (Exonúcleos)

- Eles operam baseados no conceito de multiplexação segura de hardware.
- A estratégia dos exokernels não é criar cópias (clones) da máquina, mas sim dividir de forma direta os recursos do hardware.
- Dessa forma, o hardware fica explicitamente visível para as aplicações, e cada

usuário/processo tem acesso direto a uma fração específica dos recursos.